

Data Mining w praktyce: przewidywanie zachowań klientów AdventureWorks z użyciem Pythona

Mikołaj Kubś 272662

Data Mining (eksploracja danych / knowledge discovery in databases - KDD) polega na znalezieniu nieoczywistych wzorców, trendów i zależności istniejących w dużym zbiorze danych.

Może albo pomóc opisać zbiór danych, albo przewidzieć rezultat na podstawie algorytmów uczenia maszynowego czy nawet sztucznej inteligencji.

Firmy wykorzystują te relacje do podejmowania mądrzejszych decyzji – od prognozowania sprzedaży po dogłębne zrozumienie zachowań klientów.

Zyski i wyzwania

Zyski

- Data mining bierze surowe dane i znajduje w nich pewien porządek. To może pomóc podejmować decyzje w wielu różnych korporacyjnych i przemysłowych działach, jak reklamowanie/marketing, finanse, sektor publiczny, sektor zdrowotny, HR, manufakturowanie, sprzedaż czy zarządzanie łańcuchem dostaw (SCM)
- Oszczędności: dzięki analizie danych o wydajności z wielu źródeł, można zidentyfikować problemy w procesach biznesowych. Dzięki takiej identyfikacji łatwiej będzie się ich pozbyć i zwiększyć wydajność i szybkość takich procesów.
- Data mining to wszechstronne narzędzie - każdy dział, który zbiera i analizuje dane, może z niego skorzystać.

Wyzwania

- Złożoność - data mining wymaga wiedzy z wielu działów IT - data engineering, data analysis i machine learning. Przyda się solidna wiedza z Pythona/R i SQLa. Bez dobrej znajomości machine learningu czy popełnienia błędów na poprzednich krokach, łatwo wytworzyć analizy oparte na kompletnie nieprawdziwych danych. Trzeba też uważać na nieujawnianie danych osobowych.
- Wymóg szerokiego i szczegółowego zbioru danych - trudne, zwłaszcza w dużych, starszych firmach, gdzie dane mogą być nieuporządkowane i niskiej jakości. Data mining wymaga czystego i dużego zbioru danych - najlepiej na podstawie Data Warehouse, którego zbudowanie będzie trudne, zwłaszcza z różnymi data pipelines łączące bardzo różne źródła danych.

- Niepewność - nawet dobrze przeprowadzona próba data mining może nie przynieść żadnych wartościowych rezultatów. Gorzej, jeśli zwróci rezultaty niezgodne z prawdą na podstawie różnych błędów w tym skomplikowanym procesie. Trzeba też pamiętać, że korelacja nie oznacza związku przyczynowego. Dobra, krytyczna analiza jest tu, jak zawsze, ważna.

Analiza w praktyce

Zajmijmy się praktycznym przykładem Data mining dla dobrze znanego nam zbioru danych - AdventureWorks, a dokładniej jego Data Warehouse odpowiednika - AdventureWorksDW2014. Wybrałem wersję Data Warehouse, bo przechowuje czyste i zagregowane dane, np. view o nazwie dbo.vTargetMail

Przedmiot analizy

Postaramy się dowiedzieć, co sprawia, że dany klient kupi rower? Jako cech użyjemy różnych danych demograficznych, jakie zbiera firma.

Zastosowana technika

Aby dojść do odpowiedzi, skorzystamy z potężnego i intuicyjnego algorytmu klasyfikacyjnego - Decision Tree / Drzewo Decyzyjne. Decision Tree działa jak schemat blokowy, zadając serię pytań, aby dojść do ostatecznej decyzji. Wynik można też łatwo zwizualizować i zrozumieć.

Nowoczesne narzędzia

ⓘ Important

Data mining was deprecated in SQL Server 2017 (14.x) Analysis Services and now discontinued in SQL Server 2022 (16.x) Analysis Services. Documentation is not updated for deprecated and discontinued features. To learn more, see [Analysis Services backward compatibility](#).

Niestety jak widać, należy używać starszej wersji SQL Server Analysis Services, aby wykonać data mining właśnie w Analysis Services. Dlatego skorzystałem z nowoczesnego narzędzia jakim jest Python, szeroko używanym do Machine Learningu i data engineering dzięki jego prostocie i bogactwie bibliotek. Innym językiem programowania często używanym w data science jest R. Przyda się też wiedza z SQL, którego używa się na każdym kroku w prawie każdej firmie zbierającej i analizującej swoje dane.

Krok 1: Ładowanie danych z hurtowni danych

Pierwszym krokiem w każdym projekcie analitycznym jest pozyskanie danych. Łączymy się bezpośrednio z naszą bazą danych AdventureWorksDW na SQL Server i za pomocą

zapytania SQL pobieramy potrzebne dane z widoku vTargetMail. Ten widok w wygodny sposób łączy już informacje o klientach i ich historii zakupowej. Używamy biblioteki pandas, aby załadować te dane wprost do obiektu DataFrame, która jest w zasadzie inteligentną, potężną tabelą, na której możemy operować w Pythonie.

```
In [51]: from typing import LiteralString
import pandas as pd
import pyodbc

SERVER_NAME = 'DESKTOP-2GLE46P'
DATABASE_NAME = 'AdventureWorksDW2014'

# używa Windows Authentication (Trusted_Connection=yes)
connection_string: LiteralString = f'DRIVER={{ODBC Driver 17 for SQL Server}};SE

sql_query = """
SELECT
    [CustomerKey],
    [Age],
    [MaritalStatus],
    [Gender],
    [YearlyIncome],
    [TotalChildren],
    [NumberChildrenAtHome],
    [EnglishEducation],
    [EnglishOccupation],
    [HouseOwnerFlag],
    [NumberCarsOwned],
    [CommuteDistance],
    [Region],
    [BikeBuyer]
FROM
    dbo.vTargetMail
"""

odbc_connection: pyodbc.Connection = pyodbc.connect(connection_string)

pd.set_option('display.max_columns', None)

df: pd.DataFrame = pd.read_sql(sql_query, odbc_connection)

odbc_connection.close()

print(df.head())
```

C:\Users\Jantar\AppData\Local\Temp\ipykernel_5420\1560895604.py:35: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df: pd.DataFrame = pd.read_sql(sql_query, odbc_connection)
```

	CustomerKey	Age	MaritalStatus	Gender	YearlyIncome	TotalChildren	\
0	11000	53		M	90000.0	2	
1	11001	49		S	60000.0	3	
2	11002	54		M	60000.0	3	
3	11003	51		S	70000.0	0	
4	11004	45		S	80000.0	5	

	NumberChildrenAtHome	EnglishEducation	EnglishOccupation	HouseOwnerFlag	\
0	0	Bachelors	Professional	1	
1	3	Bachelors	Professional	0	
2	3	Bachelors	Professional	1	
3	0	Bachelors	Professional	0	
4	5	Bachelors	Professional	1	

	NumberCarsOwned	CommuteDistance	Region	BikeBuyer
0	0	1-2 Miles	Pacific	1
1	1	0-1 Miles	Pacific	1
2	1	2-5 Miles	Pacific	1
3	1	5-10 Miles	Pacific	1
4	4	1-2 Miles	Pacific	1

Krok 2: Przygotowanie danych do analizy

Następnie należy dokonać preprocessingu danych.

Musimy przekonwertować string (np. MaritalStatus) na jakąś liczbową wartość. Można by każdej wartości string dla kolumny z kategoriami przypisać inną liczbę. Ale lepiej dla algorytmu maszynowego będzie stworzyć dla każdej kategorii osobną kolumnę i przypisać wartość 1/0 (tak/nie). Na przykład, dla EnglishEducation, mamy kategorie jak "Graduate Degree", "High School", "Partial College" itd. Dla każdej wartości tworzymy osobną kolumnę i tylko w (maksymalnie) jednej przypiszemy wartość 1.

Można to łatwo osiągnąć dzięki funkcji pandas.get_dummies, która dla określonych kolumn wykona ten algorytm. Dla modeli liniowych (linear/logistic regression) należy ustawić parametr drop_first na True, aby usunąć liniową zależność. Na przykład mamy kolumnę kolor, która ma wartość red, blue lub green. Jedna kolumna jest nadmiarowa, ponieważ jeśli zarówno blue jak i green są nieprawdziwe, to wiadomo, że red jest prawdziwy. Modele liniowe matematycznie zakładają, że kolumny są niezależne od siebie, więc taka zależność musi być usunięta.

Kolejnym krokiem jest usunięcie danych, których model nie powinien wykorzystywać do nauki. CustomerKey jest niepotrzebny - nie chcemy, żeby model uczył się zależności dla konkretnych klientów, więc trzeba tę kolumnę usunąć.

```
In [52]: df_processed: pd.DataFrame = df.copy()

df_processed = df_processed.drop('CustomerKey', axis=1)

df_processed = pd.get_dummies(df_processed, columns=[
    'MaritalStatus', 'Gender', 'EnglishEducation',
    'EnglishOccupation', 'CommuteDistance', 'Region'
], drop_first=True)

print("\nPrzykładowy, pierwszy wiersz:")
print(df_processed.head())
```

```
print("\nKolumny dostępne do modelowania:")  
print(df_processed.columns)
```

Przykładowy, pierwszy wiersz:

	Age	YearlyIncome	TotalChildren	NumberChildrenAtHome	HouseOwnerFlag	\
0	53	90000.0	2	0	1	
1	49	60000.0	3	3	0	
2	54	60000.0	3	3	1	
3	51	70000.0	0	0	0	
4	45	80000.0	5	5	1	

	NumberCarsOwned	BikeBuyer	MaritalStatus_S	Gender_M	\
0	0	1	False	True	
1	1	1	True	True	
2	1	1	False	True	
3	1	1	True	False	
4	4	1	True	False	

	EnglishEducation_Graduate Degree	EnglishEducation_High School	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	

	EnglishEducation_Partial College	EnglishEducation_Partial High School	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	

	EnglishOccupation_Management	EnglishOccupation_Manual	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	

	EnglishOccupation_Professional	EnglishOccupation_Skilled Manual	\
0	True	False	
1	True	False	
2	True	False	
3	True	False	
4	True	False	

	CommuteDistance_1-2 Miles	CommuteDistance_10+ Miles	\
0	True	False	
1	False	False	
2	False	False	
3	False	False	
4	True	False	

	CommuteDistance_2-5 Miles	CommuteDistance_5-10 Miles	\
0	False	False	
1	False	False	
2	True	False	
3	False	True	
4	False	False	

	Region_North America	Region_Pacific
0	False	True
1	False	True

2	False	True
3	False	True
4	False	True

Kolumny dostępne do modelowania:

```
Index(['Age', 'YearlyIncome', 'TotalChildren', 'NumberChildrenAtHome',
      'HouseOwnerFlag', 'NumberCarsOwned', 'BikeBuyer', 'MaritalStatus_S',
      'Gender_M', 'EnglishEducation_Graduate Degree',
      'EnglishEducation_High School', 'EnglishEducation_Partial College',
      'EnglishEducation_Partial High School', 'EnglishOccupation_Management',
      'EnglishOccupation_Manual', 'EnglishOccupation_Professional',
      'EnglishOccupation_Skilled Manual', 'CommuteDistance_1-2 Miles',
      'CommuteDistance_10+ Miles', 'CommuteDistance_2-5 Miles',
      'CommuteDistance_5-10 Miles', 'Region_North America', 'Region_Pacific'],
      dtype='object')
```

Krok 3: trenowanie modelu

Jak dobrze wiemy z kursu Sztuczna Inteligencja i MSID, dane zwyczajowo dzielimy na zbiór X i Y, gdzie X zawiera wszystkie dane (features) których model będzie używał do przewidzenia wartości w kolumnie Y.

Bogate w funkcje biblioteki Pythona jak sklearn pomogą łatwo podzielić dane na zbiór testowy (20%) i treningowy (reszta - 80%). Oczywiście nie można testować modelu na danych w zbiorze testowym, bo chcemy sprawdzić, czy zgeneralizował swoją wiedzę na tyle, by być użytecznym.

Sklearn zawiera już DecisionTreeClassifier - klasyfikator, którego użyjemy. Dzięki max_depth=4 będzie można jasno zwizualizować nasze dane. Ustawienie random_state na dowolną, stałą liczbę sprawi, że zawsze będziemy mieć ten sam wynik dla tych samych danych.

Następnie trenujemy model

```
In [53]: from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X: pd.DataFrame = df_processed.drop('BikeBuyer', axis=1)
Y = df_processed['BikeBuyer']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_
model = DecisionTreeClassifier(max_depth=4, random_state=42)

model.fit(X_train, Y_train)

print("\nDecisionTree model wytrenowany")
```

DecisionTree model wytrenowany

Krok 4: Ocena wydajności modelu

Sklearn oferuje gotową implementację różnych klasycznych metryk jak precision, recall czy f1-score. Wartość support oznacza, ile mamy przykładów dla danych klas. Tutaj

złożyło się tak, że jest po prawie równo klientów co kupili rower i co go nie kupili.

Accuracy to procent poprawnie odgadniętych klas. W tym wypadku jest to tylko 65% - relatywnie niska wartość, ale zawsze lepiej niż 50%. Model definitywnie się czegoś nauczył, natomiast prawdopodobnie zachowania klientów są inherentnie nieprzewidywalne lub model został słabo wytrenowany - zły model, za mała liczba danych, złe parametry, zła jakość danych (raczej nie). Zajmujemy się też fikcyjnym zbiorem danych - Adventure Works został spreparowany przez Microsoft, więc może w nim brakować dokładnego odwzorowania rzeczywistości.

W Classification Report widzimy podobne wartości dla precision, recall i f1-score. Wyniki są słabsze dla BikeBuyer = 1, zwłaszcza recall. Wartość 0.6 dla Recall dla klasy BikeBuyer = 1 oznacza, że ze wszystkich osób które kupiły rower, tylko 60% byłoby zakwalifikowane przez model jako kupujących. W sumie to na tej wartości nam najbardziej zależy - firma opierająca całą swoją strategię na tej analizie mogłaby stracić 40% potencjalnych klientów. Dla przypomnienia precyzja = $TP / (TP + FP)$, recall = $TP / (TP + FN)$, f1-score - średnia harmoniczna obu.

Jak polepszyć taki model? Zaakceptować niepewność, dodać więcej danych (cech), użyć innego algorytmu, dostroić hiperparametry.

```
In [54]: from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)

accuracy = accuracy_score(Y_test, y_pred)
print(f"\nModel Accuracy: {accuracy:.2f}")

print("\nClassification Report:")
print(classification_report(Y_test, y_pred))
```

Model Accuracy: 0.65

Classification Report:

	precision	recall	f1-score	support
0	0.64	0.69	0.66	1884
1	0.65	0.60	0.62	1813
accuracy			0.65	3697
macro avg	0.65	0.64	0.64	3697
weighted avg	0.65	0.65	0.64	3697

Krok 5: Wizualizacja

Dzięki matplotlib.pyplot można łatwo zwizualizować drzewo. Algorytm działa prosto - każdy węzeł to pytanie, jeśli odpowiedź na nie w przypadku konkretnego klienta to "tak" - idziemy w lewo, w przeciwnym wypadku w prawo. Na każdym kroku znamy prognozowaną klasę z szacowaną pewnością, ale z większą informacją w głębi drzewa model maszynowy teoretycznie ma więcej informacji i mógłby mieć większą pewność (tak też jest generalnie tutaj).

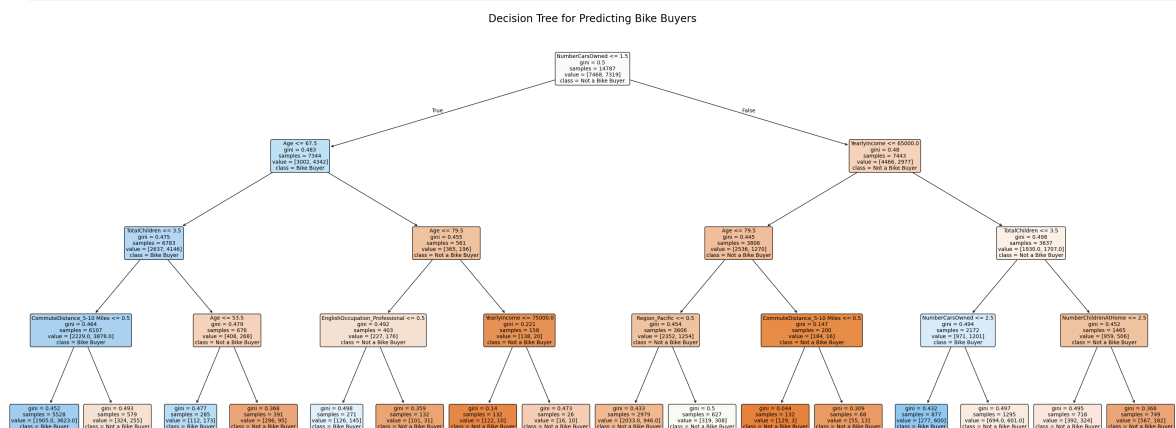
Wartość Gini (Gini impurity) to wskaźnik, jak bardzo wymieszane są dane dla klas w danym węźle drzewa.

```
In [ ]: import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

feature_names: list[str] = X.columns.tolist()
class_names: list[str] = ['Not a Bike Buyer', 'Bike Buyer']

plt.figure(figsize=(40, 15))
plot_tree(model,
          filled=True,
          feature_names=feature_names,
          class_names=class_names,
          rounded=True,
          fontsize=10)

plt.title("Decision Tree for Predicting Bike Buyers", fontsize=20)
plt.show()
```



Wnioski biznesowe

Dane mogą pomóc firmie stworzyć targetowane kampanie reklamowe do określonych grup ludzi. Dzięki sprzedaży i pozyskiwaniu danych przez korporacje takie kampanie reklamowe dobrze działają w internecie. Niestety, model radzi sobie lepiej z przewidywaniem grup które raczej nie będą zainteresowane kupnem roweru. Jest to jednak dla nas cenna informacja, dzięki data mining teraz wiemy, że osoby z liczbą samochodów większą lub równą 2, o rocznym dochodzie niższym od 65000, wieku powyżej 80 lat i typowym dystansie podróży nie będącym między 5 a 10 mil będą najmniej skłonne kupić rower i powinniśmy mniej kierować do takich grup kampanie reklamowe.

Z drugiej strony, dla osób które mają dokładnie 2 samochody, roczny dochód większy od 65000 i liczbie dzieci mniejszej od 4 będą według modelu najbardziej zainteresowane kupnem roweru firmy. Niestety, wskaźnik gini wynosi aż 0.432, co oznacza, że pewność jest bardzo niska.

Ciekawe są przykłady, gdzie 2 węzły mają bardzo różne wartości mając tego samego rodzica. Oznacza to, że wiek ≤ 53.5 jest ważnym czynnikiem predykującym kupno

roweru dla osób z liczbą dzieci co najmniej 4, wiekiem poniżej 67.5 i liczbą samochodów mniejszą od 2.

Poprawa algorytmu

W rzeczywistości, prawdziwym powodem niskiej jakości modelu jest hiperparametr `max_depth` - zwiększenie go z 4 do chociażby 15 zwiększyło accuracy z 65% do 79%.

`Max_depth = 4` został wybrany dla łatwej demonstracji wyniku działania algorytmu.

Pojawia się nowy problem - stworzenie kampanii reklamowej na podstawie 15 czynników może być trudniejsze, niż dla tylko 4.

```
In [56]: model = DecisionTreeClassifier(max_depth=15, random_state=42)

model.fit(X_train, Y_train)

print("\nDecisionTree 2 model wytrenowany")

y_pred = model.predict(X_test)

accuracy = accuracy_score(Y_test, y_pred)
print(f"\nModel Accuracy: {accuracy:.2f}")

print("\nClassification Report:")
print(classification_report(Y_test, y_pred))
```

DecisionTree 2 model wytrenowany

Model Accuracy: 0.79

Classification Report:

	precision	recall	f1-score	support
0	0.78	0.82	0.80	1884
1	0.80	0.76	0.78	1813
accuracy			0.79	3697
macro avg	0.79	0.79	0.79	3697
weighted avg	0.79	0.79	0.79	3697